

www.BrainKart.com

# Anna University

for Affiliated Engineering College - 2021 Regulation



## AID (Artificial Intelligence & Data Science Engineering)

1st Semester ▶

2nd Semester ▶

3rd Semester ▶

4th Semester ▶

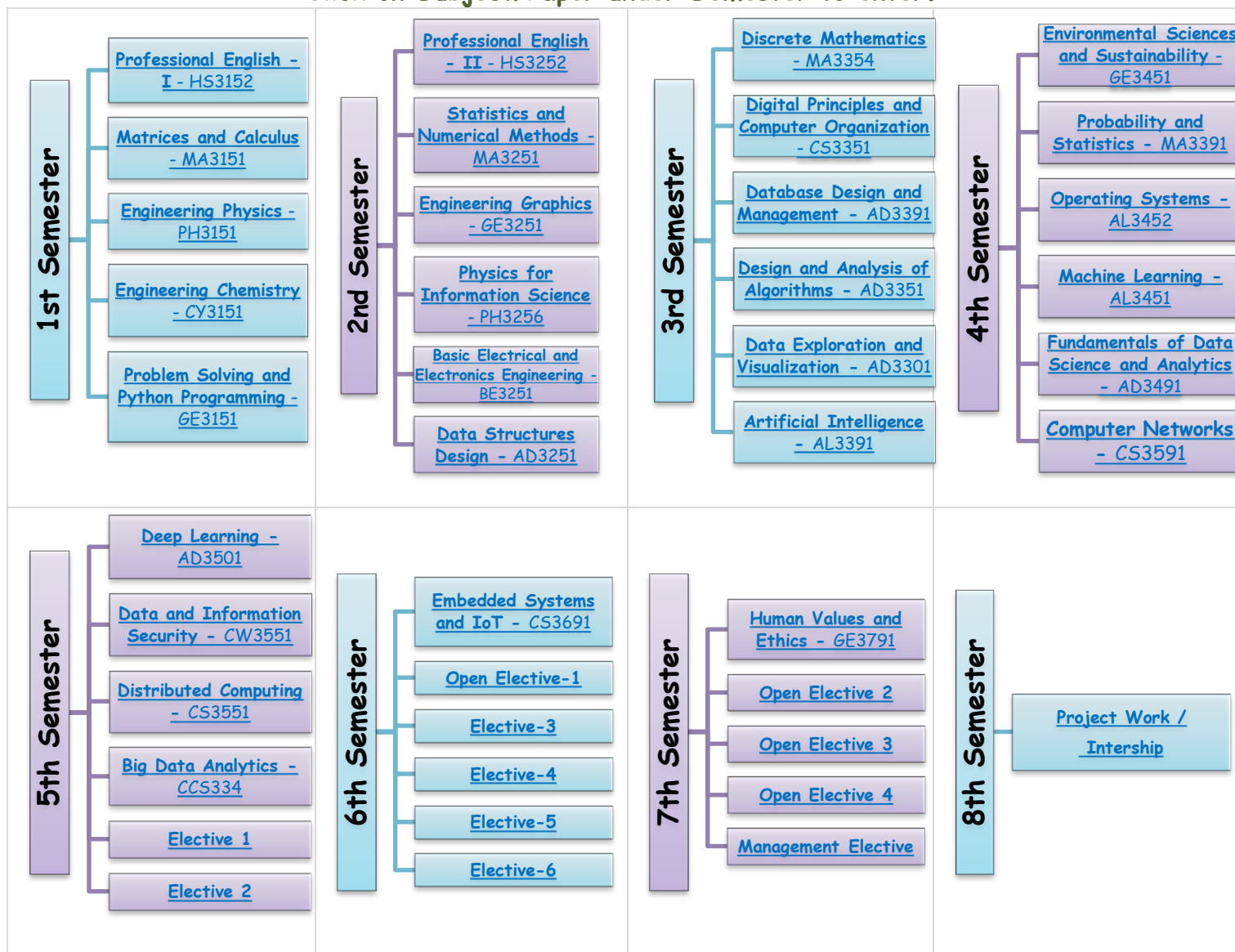
5th Semester ▶

6th Semester ▶

7th Semester ▶

8th Semester ▶

Click on Subject/Paper under Semester to enter.





**Anna University Notes**

Therithal Info  
Contains ads

3.7★

199 reviews

50K+

Downloads

3+

Rated for 3+ ©

Install



**BrainKart: Learning, Study App**

Therithal Info  
Contains ads

4.5★

160 reviews

10K+

Downloads

3+

Rated for 3+ ©

Install

**All Computer Engg Subjects - [ B.E., M.E., ]**

(Click on Subjects to enter)

|                                                                                     |                                                               |                                                                      |
|-------------------------------------------------------------------------------------|---------------------------------------------------------------|----------------------------------------------------------------------|
| <a href="#"><u>Programming in C</u></a>                                             | <a href="#"><u>Computer Networks</u></a>                      | <a href="#"><u>Operating Systems</u></a>                             |
| <a href="#"><u>Programming and Data Structures I</u></a>                            | <a href="#"><u>Programming and Data Structure II</u></a>      | <a href="#"><u>Problem Solving and Python Programming</u></a>        |
| <a href="#"><u>Database Management Systems</u></a>                                  | <a href="#"><u>Computer Architecture</u></a>                  | <a href="#"><u>Analog and Digital Communication</u></a>              |
| <a href="#"><u>Design and Analysis of Algorithms</u></a>                            | <a href="#"><u>Microprocessors and Microcontrollers</u></a>   | <a href="#"><u>Object Oriented Analysis and Design</u></a>           |
| <a href="#"><u>Software Engineering</u></a>                                         | <a href="#"><u>Discrete Mathematics</u></a>                   | <a href="#"><u>Internet Programming</u></a>                          |
| <a href="#"><u>Theory of Computation</u></a>                                        | <a href="#"><u>Computer Graphics</u></a>                      | <a href="#"><u>Distributed Systems</u></a>                           |
| <a href="#"><u>Mobile Computing</u></a>                                             | <a href="#"><u>Compiler Design</u></a>                        | <a href="#"><u>Digital Signal Processing</u></a>                     |
| <a href="#"><u>Artificial Intelligence</u></a>                                      | <a href="#"><u>Software Testing</u></a>                       | <a href="#"><u>Grid and Cloud Computing</u></a>                      |
| <a href="#"><u>Data Ware Housing and Data Mining</u></a>                            | <a href="#"><u>Cryptography and Network Security</u></a>      | <a href="#"><u>Resource Management Techniques</u></a>                |
| <a href="#"><u>Service Oriented Architecture</u></a>                                | <a href="#"><u>Embedded and Real Time Systems</u></a>         | <a href="#"><u>Multi - Core Architectures and Programming</u></a>    |
| <a href="#"><u>Probability and Queueing Theory</u></a>                              | <a href="#"><u>Physics for Information Science</u></a>        | <a href="#"><u>Transforms and Partial Differential Equations</u></a> |
| <a href="#"><u>Technical English</u></a>                                            | <a href="#"><u>Engineering Physics</u></a>                    | <a href="#"><u>Engineering Chemistry</u></a>                         |
| <a href="#"><u>Engineering Graphics</u></a>                                         | <a href="#"><u>Total Quality Management</u></a>               | <a href="#"><u>Professional Ethics in Engineering</u></a>            |
| <a href="#"><u>Basic Electrical and Electronics and Measurement Engineering</u></a> | <a href="#"><u>Problem Solving and Python Programming</u></a> | <a href="#"><u>Environmental Science and Engineering</u></a>         |





**GRACE COLLEGE OF ENGINEERING**  
(Approved by AICTE, New Delhi & Affiliated to ANNA University, Chennai)  
**MULLAKKADU, THOOTHUKUDI - 628 005**

BTech-Artificial intelligence and Data Science

Anna University Regulation: 2021

AD3391- Database Design and Management

II Year/III Semester

Hand Written Notes

Unit- III

Prepared By,

Mrs.S.Kiruthika, AP/AIDS

Unit - IV

ER and EER - to - Relational Mapping

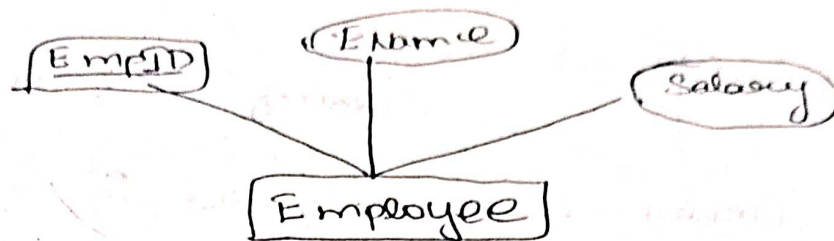
⇒ We will discuss how to map various ER model constructs to Relational Model construct.

Mapping of ~~the~~ Entity Set to Relationship:-

entity set is mapped to a relation in a straightforward way.

⇒ The primary key attribute of entity set becomes an entity of the table.

⇒ For example - Consider following ER diagram.



The converted employee table

| EmpID | EName  | Salary |
|-------|--------|--------|
| 201   | Pooram | 30000  |
| 202   | Aswini | 35000  |
| 203   | Sharda | 40000  |

Mapping Relationship sets (without constraint)

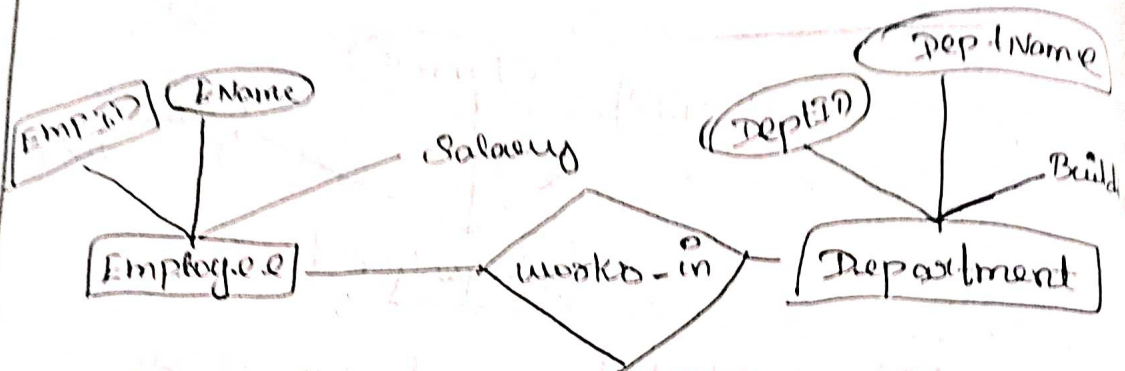
⇒ Create a table for the relationship set

⇒ Add all primary keys of the particular entity sets as fields of the table

Declare a foreign key using all key fields from the entity sets.

⇒ Declare a primary key using all key fields from the entity sets.

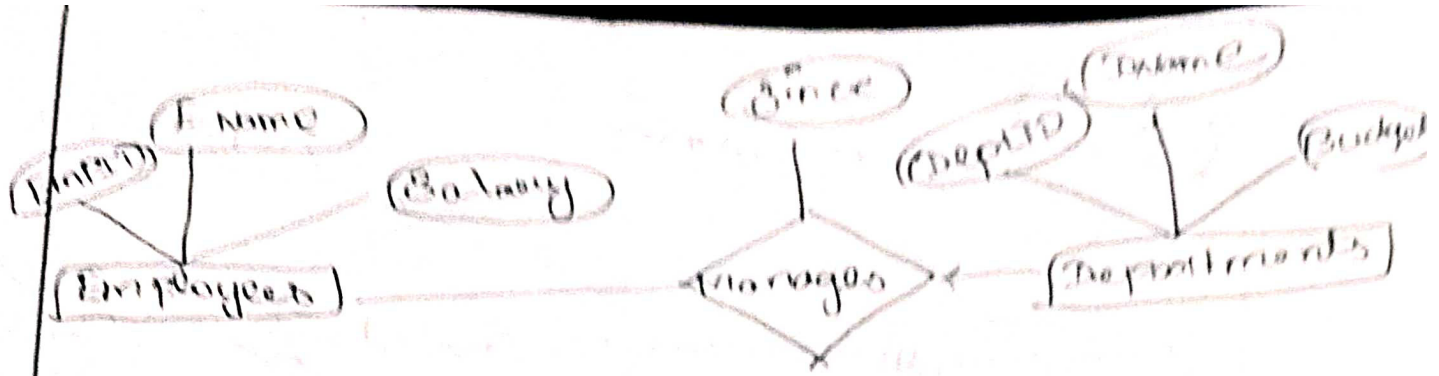
⇒ Declare foreign key constraints for all these fields from the entity sets.



### Mapping Relationship Sets

⇒ If a relationship set involves  $n$  entity sets and some  $m$  of them are linked via arrows in the ER diagram, the key for anyone of these  $m$  entity sets constitutes a key for the relation to which the relationship set is mapped.

⇒ Hence we have  $m$  candidate keys, and one of these should be designated as the primary key.



## Mapping weak Entity sets to Relational

### Mapping:-

→ A weak entity can be identified uniquely only by considering the primary key of another (owner) entity.

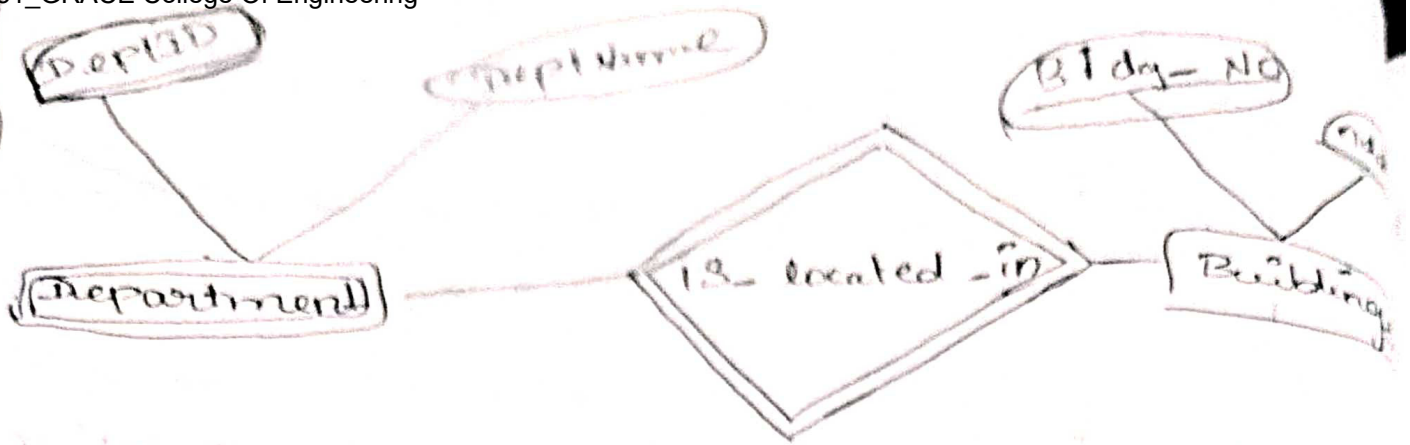
⇒ create a table for the weak entity set.

⇒ Make each attribute of the weak entity set a field of the table.

⇒ Add fields for the primary key attributes of the identifying owner.

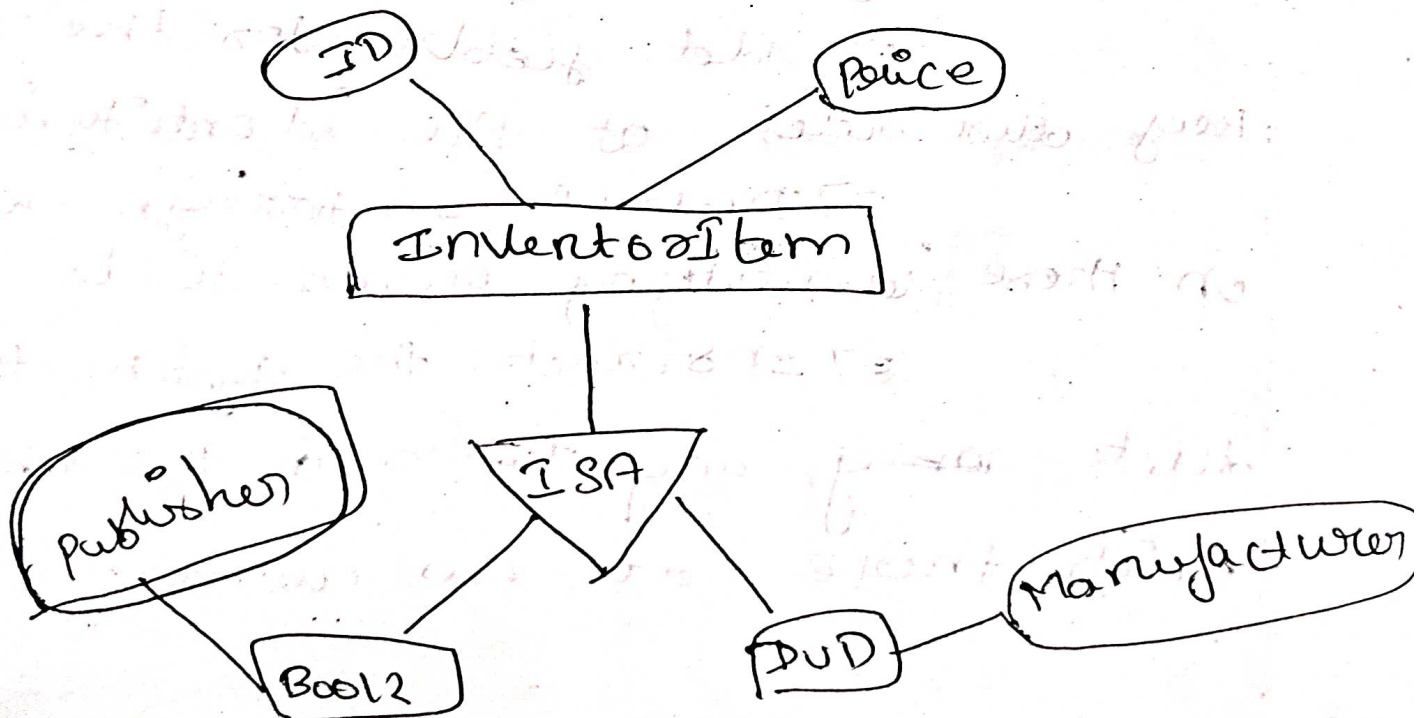
⇒ Declare a foreign key constraint on these identifying owner fields.

⇒ instruct the system to automatically delete ~~any~~ any tuples in the table for which there are no owners.



## Mapping of Specialization/Generalization CEER construct) to Relational Mapping

The Specialization / Generalization relationship can be mapped to database tables using three methods. To demonstrate the methods, we will take the -  
InventoryItem, Book DVD:



Method 1: All the entities in the relationship are mapped to individual tables

InventoryItem(ID, name)

BOOK (ID, Publisher)

DVD (ID, Manufacturer)

Method 2: Only subclasses are mapped to tables. The attributes in the superclass are duplicated in all subclasses

BOOK (ID, name, Publisher)

DVD (ID, name, Manufacturer)

Method 3: Only the superclass is mapped to a table. The attributes in the subclasses are taken to the superclass

InventoryItem(ID, name, Publisher, Manufacturer)

### update Anomalies

Definition:- Redundancy is a condition created in database in which same piece of data is held at two different places.

Redundancy is at the root of several problems associated with relational

Problems caused by redundancy

i) Redundant Storage: Some information is stored repeatedly.

ii) update anomalies: if one copy of such repeated data is updated then inconsistency is created unless all other copies are similarly updated.

iii) Insertion anomalies: Due to insertion of new record repeated information get added to the relation schema.

iv) Deletion anomalies: Due to deletion of particular record some other important information associated with the deleted record get deleted and thus we may lose some other important information from the schema.

| EmpID | ENAME | Salary | DeptID | DeptName | DeptLoc |
|-------|-------|--------|--------|----------|---------|
| 1     | AAA   | 10000  | 101    | XYZ      | Pune    |
| 2     | BBB   | 20000  | 101    | XYZ      | Pune    |
| 3     | CCC   | 30000  | 101    | XYZ      | Pune    |
| 4     | DDD   | 40000  | 102    | PQR      | Mumbai  |

Redundant

1) Redundant storage: Data that the information about DeptID, Dept Name and Dept Loc is repeated.

2) update anomalies: In above table if we change Dept Loc of Pan to Chennai, then it will result inconsistency as for DeptID 101 the Dept Loc is Pune. or otherwise we need to update multiple copies of Dept Loc from Pune to Chennai. Hence this is an update anomaly.

3) Insertion anomalies: For above table if we want to add new tuple say (5, EEE, 50000) for DeptID 101 then it will cause repeated information of (101, XYZ, Pune) will occur.

4) Deletion anomalies: For above table, if we delete a record for EmpID 4, then automatically information about the DeptID 102, Dept Name PER and Dept Loc Mumbai will get deleted and one may not be aware about DeptID 102.

## Functional Dependencies

Definition 1: A functional dependency in a relation holds if two tuples having same value of attribute A also have the same value for attribute B. It is denoted by  $A \rightarrow B$  where A is called determinant and B is called dependent.

| Roll | Name | city         |
|------|------|--------------|
| 1    | AAA  | Mumbai       |
| 2    | BBB  | Pune         |
| 3    | CCC  | Chandhrigarh |

Here

Roll  $\rightarrow$  Name hold

But

Name  $\rightarrow$  city does not hold

$\Rightarrow$  In above table, Student roll number is unique hence each student's name and city can be uniquely identified using his roll number.

$\Rightarrow$  But using name we cannot uniquely identify his/her city because there can be same names of the students. Similarly using city name we can not identify the student uniquely. As in the same

city may belong to multiple students.

### Inference Rules

$\Rightarrow$  The closure set is a set of all functional dependencies implied by a given set  $F$ . It is denoted by  $F^+$ .

$\Rightarrow$  The closure set of functional dependencies can be computed using basic three rules which are also called as Armstrong's

Axioms:-

- i) Reflexivity : If  $X \supseteq Y$ , then  $X \rightarrow Y$
  - ii) Augmentation : If  $X \rightarrow Y$ , then  $XZ \rightarrow YZ$  for any  $Z$
  - iii) Transitivity : If  $X \rightarrow Y$  and  $Y \rightarrow Z$  then  $X \rightarrow Z$
- In addition to above axioms some additional rules for computing closure of functional dependency are as follows:-

union:- If  $X \rightarrow Y$  and  $X \rightarrow Z$  then  $X \rightarrow YZ$

Decomposition:- If  $X \rightarrow YZ$ , then  $X \rightarrow Y$  and  $X \rightarrow Z$

### Example

Give FD's for relation  $R \{A, B, C, D, E, F\}$  find closure of FD set by applying Armstrong's Axioms.

Solution:

Step 1:  $A \rightarrow$  gives  $A$  itself. It is a trivial production. It is called trivial production.

$A \rightarrow B$  and  $A \rightarrow C$  hence by union rule  $A \rightarrow BC$ .

$A \rightarrow B$  and  $B \rightarrow E$ , hence by transitive rule  $A \rightarrow E$ .

$$(A)^+ = \{A, B, C, E\}$$

Step 2:  $B \rightarrow$  gives  $B$  itself.

And  $B \rightarrow E$

$$\therefore (B)^+ = \{B, E\}$$

Step 3:  $CD \rightarrow$  gives  $CD$  itself. It is trivial.

$CD \rightarrow E$ ,  $CD \rightarrow F$ , hence by union rule  $CD \rightarrow EF$ .

$$\therefore (CD)^+ = \{C, D, E, F\}$$

So by omitting trivial productions, we get.

$$\therefore F^+ = \{A \rightarrow BC, A \rightarrow E, B \rightarrow E, CD \rightarrow EF\}$$

Keys and functional Dependencies:-

i) for a given relation  $R = \{A_1, A_2, A_3 \dots\}$

$K$  is a key of  $R$  then if closure  
 $(K)^+ = \{A_1, A_2, \dots\}$  any and so subset of  $R$  i.e.  
 $\times$  Such that  $\{x\}^+ = A_1, A_2, \dots, A_n$

if, In other words there are two conditions:-

1)  $(K)^+$  contains all the attributes  
 of relation  $R$ -

2) All subset  $X$  of  $K$ ,  $(X)^+$  never  
 contains all the attributes of  $R$  i.e.  
 $(X)^+ \neq \{A_1, A_2, \dots, A_n\}$

$\Rightarrow$  If only one subset of  $R$  satisfy above  
 condition then it is known as Primary  
 key.

$\Rightarrow$  If more than one subset of  $R$   
 satisfies above condition then all these  
 subsets are recognized as candidate  
 keys. In that case one of the  
 candidate key is also considered as  
 Primary key.

$\Rightarrow$  A superset of candidate key  $K$  is known  
 as Superkey.

Given  $R = \{A, B, C, D, E, F\}$  The following  
 set  $F$  of functional dependencies  
 holds

$A \rightarrow B, A \rightarrow C, C \rightarrow H, C \rightarrow I, H \rightarrow I$   
 computes  $(AUI)^+$  is  $AUI$  candidate key?

Step 1:  $A \rightarrow B, A \rightarrow C$ , Hence add  $A, B, C$  to the set of  $AUI^+$

Step 2:  $A \rightarrow B, B \rightarrow H$ , hence add  $H$  to the set of  $AUI^+$

Step 3:  $C \rightarrow I$ , hence add  $I$  to the set of  $AUI^+$  Also add  $U$  to the set.

Thus  $(AUI)^+ = \{A, B, C, U, H, I\}$  = Relation.  
 Hence  $(AUI)^+$  is a candidate key.

### Minimal Cover

$\Rightarrow$  A set of functional dependencies  $E$  is said to be covered by  $F$  if every FD in  $E$  is also in  $F^+$ , i.e. every dependency in  $E$  can be inferred from  $F$  and vice versa.

$\Rightarrow$  Two sets of FDs  $E$  and  $F$  are equivalent if  $E^+ = F^+$ .

$\Rightarrow$  If  $E$  and  $F$  are equivalent only if both  $E^+ = F^+$

- 1)  $F$  covers  $E$
- 2)  $E$  covers  $F$

Here are two sets of FDs for R  
 $(A, B, C, D, E)$  are they equivalent

1)  $A \rightarrow B$

$AB \rightarrow C$

$D \rightarrow AC$

$D \rightarrow E$

2)  $A \rightarrow BC$

$D \rightarrow AC$

Solution: two sets of FDs are said to be equivalent if

Rule 1: If  $FD2 \supset FD1$  - that means all FDs of  $FD1$  can be derived from all the FDs of  $FD2$ .

Rule 2: If  $FD1 \supset FD2$  - that means all FDs of  $FD2$  can be derived from all the FDs of  $FD1$ .

Rule 3: If both rule 1 and rule 2 are true then  $FD1 = FD2$ .

Step 1: we will first check if all the FDs of  $FD1$  are present in  $FD2$ .

$A \rightarrow B$  is present in  $FD1$ .

$A \rightarrow BC$  is in  $FD2$ , that also means  $A \rightarrow B$  and  $A \rightarrow C$  by decomposition rule.

Hence

Similarly  $AB \rightarrow C$  is in  $FD1$

$\therefore (A)^+ = \{A, B, C\}$

$(A)^+ = \{A, B, C\}$

|                                                                          |                                                                                     |
|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| $\therefore (CD)^+ = \{A, B, C\}$                                        | As "Decomposition rule"                                                             |
| $D \rightarrow AC$                                                       | $D \rightarrow A, D \rightarrow C$                                                  |
| $\therefore D \rightarrow A, D \rightarrow C$ by decomposition rule      | As $A \rightarrow B$ , then by transitivity rule $D \rightarrow B, D \rightarrow C$ |
| then $D \rightarrow A, A \rightarrow B$ and                              | $\therefore (CD)^+ = \{A, B, C, D, E\}$                                             |
| $D \rightarrow A, D \rightarrow B, D \rightarrow C$ by transitivity rule |                                                                                     |
| $D \rightarrow E$ is given                                               |                                                                                     |
| $\therefore (CD)^+ = \{A, B, C, D, E\}$                                  |                                                                                     |

Step 2:- we will first check if all the FDs + FD2 are present in FD1

|                                        |                                         |
|----------------------------------------|-----------------------------------------|
| $\therefore (A)^+ = \{A, B, C\}$       | $(A)^+ = \{A, B, C\}$                   |
| $\therefore (AC)^+ = \{A, B, C\}$      | $\therefore (CD)^+ = \{A, B, C, D, E\}$ |
| $\therefore (D)^+ = \{A, B, C, D, E\}$ |                                         |

Minimal Cases:

Definition: A minimal cover for a set  $F$  of FDs is a set  $G$  of FDs such that

- 1) Every dependency in  $G$  is of the form  $X \rightarrow A$ , where  $A$  is a single attribute
- 2) The closure  $F^+$  is equal to the closure  $G^+$

~~3) It is irreducible~~

### Concept of extraneous attributes:-

Definition:- An attribute of a functional dependency is said to be extraneous if we can remove it without changing the closure of the set of functional dependencies.

Consider a set  $F$  of functional dependencies and the functional dependency  $\alpha \rightarrow B$  in  $F$ .

$\Rightarrow$  Attribute  $A$  is extraneous in  $\alpha$  if  $A \in \alpha$  and  $F$  logically implies

$$(F - \{\alpha \rightarrow B\}) \cup \{(\alpha - A) \rightarrow B\}$$

$\Rightarrow$  Attribute  $A$  is extraneous in  $B$  if  $A \in B$  and the set of functional dependencies

$$(F - \{\alpha \rightarrow B\}) \cup \{\alpha \rightarrow (B - A)\}$$

logically implies  $F$ .

For Set of functional dependencies

$F_c \subseteq F$

Repeat:

use the union rule to replace any dependencies in  $F_c$  of the form:

$\alpha_1 \rightarrow \beta_1$  and  $\alpha_1 \rightarrow \beta_2$  and  $\alpha_1 \rightarrow \beta_2$

Find a functional dependency  $\alpha \rightarrow \beta$  in  $F_c$  with an extraneous attribute either in  $\alpha$  or in  $\beta$ .

If an extraneous attribute is found, delete it from  $\alpha \rightarrow \beta$  in  $F_c$  until

properties of Relational Decomposition:

> Decomposition is the process of breaking an one table into multiple tables.

Formal definition of decomposition is-

A decomposition of relation schema  $R$  into two relation schema that each contain a subset of attributes

of  $R$  and together include all attributes of  $R$  by storing projections of the instance.

For example - consider the following table.

Employee - Department

| Eid  | Ename | Age | City      | Salary | Deptid | Dept Name      |
|------|-------|-----|-----------|--------|--------|----------------|
| E001 | ABC   | 29  | Pune      | 20000  | D001   | Finance        |
| E002 | PQR   | 30  | Pune      | 30000  | D002   | Marketing      |
| E003 | LMN   | 25  | Mumbai    | 5000   | D003   | Sales          |
| E004 | XYZ   | 24  | Mumbai    | 4000   | D004   | Marketing      |
| E005 | STU   | 32  | Hyderabad | 25000  | D005   | Human Resource |

→ We can decompose the above relation schema into two relation schemas.  
 Employee (Eid, Ename, Age, City, Salary)  
 and Department (Deptid, Eid, DeptName)  
 Employee Table

| Eid  | Ename | Age | City      | Salary |
|------|-------|-----|-----------|--------|
| E001 | ABC   | 29  | Pune      | 20000  |
| E002 | PQR   | 30  | Pune      | 30000  |
| E003 | LMN   | 25  | Mumbai    | 5000   |
| E004 | XYZ   | 24  | Mumbai    | 4000   |
| E005 | STU   | 32  | Hyderabad | 25000  |

Department Table:-

| Deptid | Eid  | Dept Name |
|--------|------|-----------|
| D001   | E001 | Finance   |

| DOO 2 | EDD 3 | Product        |
|-------|-------|----------------|
| DOO 3 | EDD 3 | Sales          |
| DOO 4 | EDD 4 | Marketing      |
| DOO 5 | EDD 5 | Human Resource |

⇒ The decomposition is used for eliminating redundancy.

⇒ For example: consider following relation Schema R in which we assume that the grade determines the salary, the redundancy is caused.

Schema R

| Name | eid | deptname  | grade | Salary |
|------|-----|-----------|-------|--------|
| AAA  | 121 | Accounts  | 2     | 800    |
| AAA  | 132 | Sales     | 3     | 700    |
| BBB  | 101 | Marketing | 4     | 700    |
| CCC  | 106 | Purchase  | 2     | 800    |

Hence the above table can be decomposed into two Schema S and T as follows.

Scheme C

| NAME | id  | department | grade | salary | Salary |
|------|-----|------------|-------|--------|--------|
| AAA  | 101 | Account    | 2     | 2      | 8000   |
| AAA  | 122 | Bank       | 3     | 3      | 7000   |
| BBB  | 101 | Marketing  | 4     | 4      | 7000   |
| CCC  | 106 | Purchase   | 2     | 2      | 8000   |

Problems related to decomposition

1) Some queries become more expensive  
 2) given instances of the decomposed relations, we may not be able to reconstruct the corresponding instance of the original relation.

3) there may be loss of information during decomposition.

Properties associated with decomposition  
 There are two properties associated with decomposition and those are.

1) Loss- less join or non loss decomposition  
 when all information found in the original database is preserved after decomposition, we call it as loss less or non loss decomposition.

2) Lossy decomposition: It is a property of relation the constraints on the original table can be maintained by simply enforcing some constraints on each of the smaller relations.

Lossy Join:-

i) Union of attributes of  $R_1$  and  $R_2$  must be equal to attributes of  $R$  - each attribute of  $R$  must be either in  $R_1$  or in  $R_2$ .

$$Att(R_1) \cup Att(R_2) = Att(R)$$

ii) Intersection of attributes of  $R_1$  and  $R_2$  must not be NULL.

$$Att(R_1) \cap Att(R_2) \neq \emptyset$$

iii) Common attribute must be a key for at least one relation ( $R_1$  or  $R_2$ )

$$Att(R_1) \cap Att(R_2) \rightarrow Att(R_1)$$

Consider the following relation  $R(A, B, C, D)$  and FDs  $A \rightarrow BC$ , is the decomposition of  $R$  into  $R_1(A, B, C)$ ,  $R_2(A, D)$

check if the decomposition is lossless join or not.

Step 1: Here all attributes  $(R_1) = \{A, B, C, D\}$  are in  $R_2$ .  
 $R_1(A, B, C) \cup R_2(A, D) = (A, B, C, D) \in R$   
 Thus, first condition gets satisfied.

Step 2: Here  $R_1 \cap R_2 = \{A, B\}$ . Here all attributes  $(R_2)$  are in  $R_1$ . Thus the second condition gets satisfied.

Step 3:  $R_1 \cap R_2 = \{A, B\}$ . Here  $(A, B) \in R$ . Thus the third condition gets satisfied.

Dependency Preservation:-

Definition :- A decomposition  $D = \{R_1, R_2, R_3, \dots, R_n\}$  of  $R$  is dependency preserving for a set  $F$  of functional dependencies if  $(F_1 \cup F_2 \cup \dots \cup F_n) = F$ .

If decomposition is not dependency-preserving, some dependency is lost in the decomposition.

Normalization (upto BCNF)

Normalization is the process of reorganizing data in a database so that it meets two basic requirements:

main requirements:-

- 1) There is no redundancy of (all data is stored in only one place) and
  - 2) data dependencies are verified (all related data items are stored together)
  - 3) The normalization is important because it allows database to take up less space.
  - 4) It also help in increasing the performance.
- First Normal form:-

- i) It should only have single (atomic) valued attributes / columns.
- ii) values stored in a column should be of the same domain.
- iii) All the ~~the~~ columns in a table should have unique names.
- iv) And the order in which data is stored, does not matter.

Student .

|   |     |       |
|---|-----|-------|
| 1 | AAA | 1111  |
| 2 | BBB | 22222 |
| 3 | CCC | 44444 |
|   |     | 55555 |

As there are multiple values of phone number for sid 1 and 3, the above table is not in 1NF. We can make it in 1NF. The conversion is as follows.

| sid | Sname | phone |
|-----|-------|-------|
| 1   | AAA   | 11111 |
| 1   | AAA   | 22222 |
| 2   | BBB   | 33333 |
| 3   | CCC   | 44444 |
| 3   | CCC   | 55555 |

## Second Normal Form

Before understanding the second normal form let us first discuss the concept of partial

functional dependency and prime and non prime attributes

concept of partial functional dependency.

Partial dependency means that non prime attribute is functionally dependent on part of a candidate key.

consider a relation  $R(A, B, C, D)$  with functional dependency.

$$\{AB \rightarrow CD, A \rightarrow C\}$$

Here  $(AB)$  is a candidate key because  $(AB)^+ = \{AB, CD\} = \{R\}$ .

Prime and Non Prime attributes

Prime attributes An attribute, which is a part of the candidate key is known as a prime attribute.

Non - Prime attribute: An attribute, which is not a part of the prime key, is said to be a non - prime attribute.

1) It should be in the first normal form.  
 2) It should not have partial functional dependency.

Student - course.

| Sid | Sname | Cid | Cname |
|-----|-------|-----|-------|
| 1   | AAA   | 101 | C     |
| 2   | BBB   | 102 | C++   |
| 3   | CCC   | 101 | C     |
| 4   | DDD   | 103 | Java  |

Step 1: The above table is in 1NF.

Step 2: Here Sname and Sid are associated similarly cid and cname are associated with each other.

Now if we delete a record with Sid then automatically the course C++ will also get deleted. Thus,

Sid  $\rightarrow$  Sname or ~~id~~ cid  $\rightarrow$  cname is a partial functional dependency, because {Sid, cid} should be essentially a candidate key for

| id | name | cid |
|----|------|-----|
| 1  | AN A | 101 |
| 2  | BB B | 102 |
| 3  | CC C | 101 |
| 4  | DD D | 103 |

course

| cid | cname |
|-----|-------|
| 101 | C     |
| 102 | C++   |
| 101 | C     |
| 103 | Jave. |

Third Normal Form:

Before understanding the third normal form let us first discuss the concept of transitive dependency, super key and candidate key.

concept of Transitive Dependency.

$\Rightarrow$  A functional dependency is said to be transitive if it is indirectly formed by two functional dependencies.

$x \rightarrow z$  is a transitive dependency if the following functional dependencies hold true:

$$x \rightarrow y$$

$$y \rightarrow z$$

concept of super key and candidate key.

**Super key**:- A Super key is a set of one or more (attributes) to uniquely identify rows in a table.

**candidate key**: The minimal set of attribute which can uniquely identify a tuple is known as candidate key.

| RegID | Roll NO | Sname |
|-------|---------|-------|
| 101   | 1       | AAA   |
| 102   | 2       | BBB   |
| 103   | 3       | CCC   |
| 104   | 4       | DDD   |

Super keys

{ RegID }

{ RegID, Roll No }

{ RegID, Name }

{ RegID, Roll No, Name }

{ RegID, Roll No, Name }

Candidate keys

{ RegID }

{ Roll No }

Third Normal Form

i) It is in the second normal form (i.e) it does not have partial functional dependency)

ii) It doesn't have ~~any~~ transitive dependency

at least one of the following conditions hold:

i)  $\lambda$  is a Super key of table

ii)  $\lambda$  is a prime attribute of table

For example: Consider following table  
Student details

| id | name | zipcode | cityname | state       |
|----|------|---------|----------|-------------|
| 1  | AAA  | 11111   | Pune     | Maharashtra |
| 2  | BBB  | 22222   | Mumbai   | Maharashtra |
| 3  | CCC  | 33333   | Chennai  | Tamil Nadu  |
| 4  | DDD  | 44444   | Delhi    | Rajasthan   |
| 5  | EEE  | 55555   | Mumbai   | Maharashtra |

Super keys :  $\{sid\}$ ,  $\{sid, name\}$ ,  $\{sid, zipcode\}$ ,  $\{sid, cityname\}$ ,  $\{sid, state\}$ ,  $\{sid, zipcode, cityname\}$  - - and so on.

Candidate key :  $\{sid\}$

Non-Prime attributes :  $\{name, zipcode, cityname, state\}$

The dependencies can be denoted as

$sid \rightarrow name$

$sid \rightarrow zipcode$

$zipcode \rightarrow cityname$

$cityname \rightarrow state$

Boyce/Codd Normal Form (BCNF)

→ Boyce and Codd Normal Form is a higher version of the third Normal Form.

→ A 3NF table which does not have multiple overlapping candidate key is said to be in BCNF.

i) R must be in 3rd Normal Form

(i) For each functional dependency  $(X \rightarrow Y)$ ,  $X$  should be a super key. In simple words if  $Y$  is a prime attribute then  $X$  can not be non prime attribute.

enrollment table

| sid | course | Teacher |
|-----|--------|---------|
| 1   | C      | Ankita  |
| 1   | Java   | Poonam  |
| 2   | C      | Ankita  |
| 3   | C++    | Supriya |
| 4   | C      | Archana |

From above table following observations can be made:-

- ⇒ One student can enroll for multiple courses. For example student with sid = 1 can enroll for C as well as Java.
- ⇒ For each course, a teacher is assigned to the student.
- ⇒ The candidate key for above table can be (sid, course) because using these two columns we can find

The above table holds following dependencies,

⇒ (sid, Course) → Teacher

⇒ Teacher → course

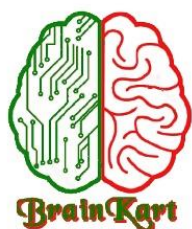
⇒ To convert the above table to BCNF we must decompose above table into student and course tables.

Student

| Sid | Teacher |
|-----|---------|
| 1   | Ankita  |
| 1   | Pooam   |
| 2   | Ankita  |
| 2   | Supriya |

| Teacher | Course |
|---------|--------|
| Ankita  | C      |
| Poonam  | Java   |
| Ankita  | C      |
| Supriya | C++    |
| Ashwina | C      |

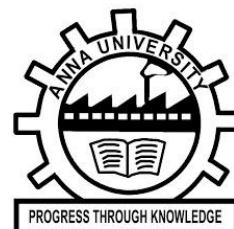
now the table is in BCNF



www.BrainKart.com

# Anna University

for Affiliated Engineering College - 2021 Regulation



## AID (Artificial Intelligence & Data Science Engineering)

1st Semester ▶

2nd Semester ▶

3rd Semester ▶

4th Semester ▶

5th Semester ▶

6th Semester ▶

7th Semester ▶

8th Semester ▶

Click on Subject/Paper under Semester to enter.

### 1st Semester

[Professional English - I - HS3152](#)

[Matrices and Calculus - MA3151](#)

[Engineering Physics - PH3151](#)

[Engineering Chemistry - CY3151](#)

[Problem Solving and Python Programming - GE3151](#)

### 2nd Semester

[Professional English - II - HS3252](#)

[Statistics and Numerical Methods - MA3251](#)

[Engineering Graphics - GE3251](#)

[Physics for Information Science - PH3256](#)

[Basic Electrical and Electronics Engineering - BE3251](#)

[Data Structures Design - AD3251](#)

### 3rd Semester

[Discrete Mathematics - MA3354](#)

[Digital Principles and Computer Organization - CS3351](#)

[Database Design and Management - AD3391](#)

[Design and Analysis of Algorithms - AD3351](#)

[Data Exploration and Visualization - AD3301](#)

[Artificial Intelligence - AL3391](#)

### 4th Semester

[Environmental Sciences and Sustainability - GE3451](#)

[Probability and Statistics - MA3391](#)

[Operating Systems - AL3452](#)

[Machine Learning - AL3451](#)

[Fundamentals of Data Science and Analytics - AD3491](#)

[Computer Networks - CS3591](#)

### 5th Semester

[Deep Learning - AD3501](#)

[Data and Information Security - CW3551](#)

[Distributed Computing - CS3551](#)

[Big Data Analytics - CCS334](#)

[Elective 1](#)

[Elective 2](#)

### 6th Semester

[Embedded Systems and IoT - CS3691](#)

[Open Elective-1](#)

[Elective-3](#)

[Elective-4](#)

[Elective-5](#)

[Elective-6](#)

### 7th Semester

[Human Values and Ethics - GE3791](#)

[Open Elective 2](#)

[Open Elective 3](#)

[Open Elective 4](#)

[Management Elective](#)

### 8th Semester

[Project Work / Internship](#)



**Anna University Notes**

Therithal Info  
Contains ads

3.7★

199 reviews

50K+

Downloads

3+

Rated for 3+ ©

Install



**BrainKart: Learning, Study App**

Therithal Info  
Contains ads

4.5★

160 reviews

10K+

Downloads

3+

Rated for 3+ ©

Install

**All Computer Engg Subjects - [ B.E., M.E., ]**

(Click on Subjects to enter)

|                                                                                     |                                                               |                                                                      |
|-------------------------------------------------------------------------------------|---------------------------------------------------------------|----------------------------------------------------------------------|
| <a href="#"><u>Programming in C</u></a>                                             | <a href="#"><u>Computer Networks</u></a>                      | <a href="#"><u>Operating Systems</u></a>                             |
| <a href="#"><u>Programming and Data Structures I</u></a>                            | <a href="#"><u>Programming and Data Structure II</u></a>      | <a href="#"><u>Problem Solving and Python Programming</u></a>        |
| <a href="#"><u>Database Management Systems</u></a>                                  | <a href="#"><u>Computer Architecture</u></a>                  | <a href="#"><u>Analog and Digital Communication</u></a>              |
| <a href="#"><u>Design and Analysis of Algorithms</u></a>                            | <a href="#"><u>Microprocessors and Microcontrollers</u></a>   | <a href="#"><u>Object Oriented Analysis and Design</u></a>           |
| <a href="#"><u>Software Engineering</u></a>                                         | <a href="#"><u>Discrete Mathematics</u></a>                   | <a href="#"><u>Internet Programming</u></a>                          |
| <a href="#"><u>Theory of Computation</u></a>                                        | <a href="#"><u>Computer Graphics</u></a>                      | <a href="#"><u>Distributed Systems</u></a>                           |
| <a href="#"><u>Mobile Computing</u></a>                                             | <a href="#"><u>Compiler Design</u></a>                        | <a href="#"><u>Digital Signal Processing</u></a>                     |
| <a href="#"><u>Artificial Intelligence</u></a>                                      | <a href="#"><u>Software Testing</u></a>                       | <a href="#"><u>Grid and Cloud Computing</u></a>                      |
| <a href="#"><u>Data Ware Housing and Data Mining</u></a>                            | <a href="#"><u>Cryptography and Network Security</u></a>      | <a href="#"><u>Resource Management Techniques</u></a>                |
| <a href="#"><u>Service Oriented Architecture</u></a>                                | <a href="#"><u>Embedded and Real Time Systems</u></a>         | <a href="#"><u>Multi - Core Architectures and Programming</u></a>    |
| <a href="#"><u>Probability and Queueing Theory</u></a>                              | <a href="#"><u>Physics for Information Science</u></a>        | <a href="#"><u>Transforms and Partial Differential Equations</u></a> |
| <a href="#"><u>Technical English</u></a>                                            | <a href="#"><u>Engineering Physics</u></a>                    | <a href="#"><u>Engineering Chemistry</u></a>                         |
| <a href="#"><u>Engineering Graphics</u></a>                                         | <a href="#"><u>Total Quality Management</u></a>               | <a href="#"><u>Professional Ethics in Engineering</u></a>            |
| <a href="#"><u>Basic Electrical and Electronics and Measurement Engineering</u></a> | <a href="#"><u>Problem Solving and Python Programming</u></a> | <a href="#"><u>Environmental Science and Engineering</u></a>         |

